

Python basico

Mini guia personal de Python con ejemplos ordenados, explicacion breve y resultado esperado en consola.

Nota: en el archivo original, la seccion de comprehensions usa `claves`, `valores` y `numeros` sin definir.
En esta guia se agregan valores de ejemplo para que el codigo sea ejecutable.

Indice

1. `print()`
2. Variables
3. Tipos de datos
4. Operaciones matematicas
5. Strings
6. `input()`
7. Condicionales
8. `for`
 - Comprehensions
9. `while`
10. Funciones
11. Listas
12. Tuplas
13. Sets
14. Diccionarios
15. Librerias
16. Modularizacion
 - Refactorizacion
17. Experiencia de usuario
18. Programacion orientada a objetos

1. `print()`

`print()` sirve para mostrar informacion en consola.

```
print("Hola Mundo")  
# Entrega Hola Mundo
```

2. Variables

Una variable guarda un valor para usarlo despues.

```
nombre = "Sebastian"  
edad = 30  
altura = 1.76
```

```
es_programador = False

print(nombre)
print(edad)
print(altura)
print(es_programador)
```

Resultado:

```
Sebastian
30
1.76
False
```

3. Tipos de datos

`type()` muestra el tipo de dato de una variable.

```
print(type(nombre))
print(type(edad))
print(type(altura))
print(type(es_programador))
```

Resultado:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

4. Operaciones matematicas

Python permite hacer operaciones matematicas directamente.

```
import math

a = 10
b = 3

print("Suma:", a + b)
print("Resta:", a - b)
print("Multiplicacion:", a * b)
print("Division:", a / b)
print("Potencia:", a ** b)
print("Raiz cuadrada:", math.sqrt(a))
```

```
print("Division entera:", a // b)
print("Modulo:", a % b)
print("Redondeo:", round(3.14159, 2))
print("Redondeo hacia arriba:", math.ceil(3.14159))
print("Redondeo hacia abajo:", math.floor(3.14159))
```

Resultado:

```
Suma: 13
Resta: 7
Multiplicacion: 30
Division: 3.3333333333333335
Potencia: 1000
Raiz cuadrada: 3.1622776601683795
Division entera: 3
Modulo: 1
Redondeo: 3.14
Redondeo hacia arriba: 4
Redondeo hacia abajo: 3
```

5. Strings

Los strings son textos. Se pueden unir, transformar y medir.

```
nombre = "Sebastian"

print("Hola " + nombre)
print(f"Hola {nombre}")
print(nombre.upper())
print(nombre.lower())
print(len(nombre))
```

Resultado:

```
Hola Sebastian
Hola Sebastian
SEBASTIAN
sebastian
9
```

Metodos utiles de strings:

Metodo	Para que sirve
<code>capitalize()</code>	Pone la primera letra en mayuscula.

Metodo	Para que sirve
<code>title()</code>	Pone cada palabra con inicial mayuscula.
<code>strip()</code>	Quita espacios al inicio y al final.
<code>replace()</code>	Reemplaza texto.
<code>find()</code>	Busca la posicion de un texto.
<code>split()</code>	Divide un string en una lista.

Ejemplos parecidos en Django o ARCA:

```
transaction.save()
serializer.is_valid()
queryset.filter(...)
```

6. `input()`

`input()` permite pedir informacion al usuario. En el archivo original esta comentado para que el programa no se detenga esperando una respuesta.

```
nombre_ingresado = input("Como te llamas? ")
print(f"Hola {nombre_ingresado}")

numero_ingresado = int(input("Ingrese un numero: "))
print(numero_ingresado * 2)
```

Ejemplo de resultado si el usuario escribe `Sebastian` y luego `5`:

```
Como te llamas? Sebastian
Hola Sebastian
Ingrese un numero: 5
10
```

7. Condicionales

`if`, `elif` y `else` permiten tomar decisiones.

```
numero = 8

if numero > 0:
    print("Positivo")
elif numero == 0:
    print("Es cero")
```

```
else:  
    print("Negativo")
```

Resultado:

Positivo

Operadores de comparacion:

Operador	Significado
<code>==</code>	Igual a
<code>!=</code>	Distinto de
<code>></code>	Mayor que
<code><</code>	Menor que

Operadores logicos:

Operador	Significado
<code>and</code>	Y
<code>or</code>	O
<code>not</code>	No

8. `for`

`for` sirve para recorrer elementos de una secuencia.

Si quieres...	Usa
Repetir hasta que ocurra algo	<code>while</code>
Recorrer una lista	<code>for elemento in lista</code>
Contar de 0 a N	<code>for i in range(10)</code>
Recorrer un texto letra por letra	<code>for letra in texto</code>
Recorrer un diccionario	<code>for clave, valor in persona.items()</code>
Obtener indice y valor	<code>enumerate()</code>
Recorrer dos listas al mismo tiempo	<code>zip()</code>
Salir antes de terminar	<code>break</code>
Saltar una iteracion	<code>continue</code>
Usar un bucle dentro de otro	Bucle anidado

range()

```
for i in range(10, 1, -1):  
    print(i)
```

```
""  
resultado  
10  
9  
8  
7  
6  
5  
4  
3  
2  
""
```

Recorrer un texto

```
texto = "Hola"  
  
for letra in texto:  
    print(letra)
```

```
""  
Resultado  
H  
o  
l  
a  
""
```

Recorrer un diccionario

```
persona = {  
    "nombre": "Sebastian",  
    "edad": 30,  
}  
  
for clave, valor in persona.items():  
    print(f"{clave}: {valor}")
```

```
""  
Resultado:  
nombre: Sebastian
```

```
edad: 30
"""
```

enumerate()

`enumerate()` entrega posicion y valor.

```
frutas = ["Manzana", "Pera", "Kiwi"]

for posicion, fruta in enumerate(frutas):
    print(posicion, fruta)

"""
R:
0 Manzana
1 Pera
2 Kiwi
"""
```

zip()

`zip()` combina listas y entrega elementos correspondientes.

```
nombres = ["Ana", "Luis", "Pedro"]
edades = [20, 30, 40]

for nombre, edad in zip(nombres, edades):
    print(nombre, edad)

"""
R:
Ana 20
Luis 30
Pedro 40
"""
```

break

`break` termina el bucle antes de tiempo.

```
for i in range(10):
    if i == 5:
        break
    print(i)

"""
R:
```

```
0
1
2
3
4
"""
```

continue

`continue` salta una iteracion y sigue con la siguiente.

```
for i in range(10):
    if i == 5:
        continue
    print(i)

"""
R:
0
1
2
3
4
6
7
8
9
"""
```

Bucle anidado

Un bucle anidado es un bucle dentro de otro.

```
for fila in range(3):
    for columna in range(4):
        print(fila, columna)

"""
R:
0 0
0 1
0 2
0 3
1 0
1 1
1 2
1 3
2 0
2 1
"""
```



```
2 2
2 3
"""
```

Comprehensions

Las comprehensions permiten crear listas o diccionarios de forma mas corta.

List comprehension

```
cuadrados = []

for i in range(10):
    cuadrados.append(i ** 2)

# todo lo anterior pasa a ser:
cuadrados = [i ** 2 for i in range(10)]

print(cuadrados)

# Resultado [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Dict comprehension

```
claves = ["nombre", "edad", "pais"]
valores = ["Sebastian", 30, "Chile"]

diccionario = {k: v for k, v in zip(claves, valores)}

print(diccionario)

# {'nombre': 'Sebastian', 'edad': 30, 'pais': 'Chile'}
```

Condicional dentro de una list comprehension

```
numeros = [1, 2, 3, 4, 5, 6]

resultado = ["Par" if numero % 2 == 0 else "Impar" for numero in numeros]

print(resultado)

# Resultado ['Impar', 'Par', 'Impar', 'Par', 'Impar', 'Par']
```

Filtrar elementos

```
numeros = [1, 2, 3, 4, 5, 6]

pares = [numero for numero in numeros if numero % 2 == 0]

print(pares)

# Resultado [2, 4, 6]
```

9. while

while repite instrucciones mientras una condicion sea verdadera.

```
contador = 1

while contador <= 3:
    print(contador)
    contador += 1
```

Resultado:

```
1
2
3
```

10. Funciones

Una funcion agrupa instrucciones para reutilizarlas

```
def sumar(x, y): # parámetros
    return x + y

a = sumar(5, 7) # Argumentos
print(a)

#R: 12
```

Parametros nombrados

Los parametros nombrados permiten indicar que valor recibe cada parametro.

```
def dividir(a, b):
    print(a / b)
```

```
dividir(a=10, b=2)
dividir(b=2, a=10)
# En ambas el resultado es 5.0
```

Parametros por defecto

Un parametro puede tener un valor por defecto. Ese valor se usa si no se entrega un argumento.

```
def saludar(nombre="Usuario"):
    print(f"Hola {nombre}")

saludar() # Hola Usuario
saludar("Sebastian") # Hola Sebastian
```

return

`print()` muestra informacion en consola.

`return` devuelve un valor para poder guardarlo o usarlo despues.

Ejemplo usando `print()`:

```
def cuadrado(x):
    print(x ** 2)

a = cuadrado(5) #25
print(a) #None
```

Ejemplo usando `return`:

```
def cuadrado(x):
    return x ** 2

a = cuadrado(5)
print(a) #25
```

Ahora la funcion devuelve el valor y se puede guardar en la variable `resultado`.

Multiples retornos

Una funcion puede devolver mas de un valor.

```
def operaciones(numero):  
    cuadrado = numero ** 2  
    cubo = numero ** 3  
    return cuadrado, cubo  
  
c2, c3 = operaciones(4)  
  
print(c2)#16  
print(c3)#64  
#
```

Esto se llama desempaquetado. Cada valor retornado se guarda en una variable.

Funciones que reciben listas

Una funcion puede recibir una lista como argumento.

```
def promedio(lista):  
    total = sum(lista)  
    cantidad = len(lista)  
    return total / cantidad  
  
notas = [6, 5, 7]  
resultado = promedio(notas)  
  
print(resultado) #6.0
```

Funciones que reciben diccionarios

Una funcion tambien puede recibir un diccionario.

```
def mostrar_productos(diccionario):  
    for producto, precio in diccionario.items():  
        print(f"{producto}: ${precio}")  
  
productos = {  
    "Pan": 1200,  
    "Leche": 1000,  
    "Queso": 3500  
}  
  
mostrar_productos(productos)  
"""  
Imprime  
Pan: $1200  
Leche: $1000
```

```
Queso: $3500
"""
```

Funciones como argumentos

En Python una funcion tambien es un objeto. Por eso se puede enviar una funcion como argumento a otra funcion.

```
def operar(lista, funcion):
    return funcion(lista)

numeros = [10, 20, 30]

print(operar(numeros, sum)) #60
print(operar(numeros, len)) #3
```

Se escribe `sum` y no `sum()` porque queremos entregar la funcion, no ejecutarla inmediatamente.

`*args`

`*args` permite recibir una cantidad variable de argumentos.

Sirve cuando no sabemos cuantos valores se van a entregar.

```
def sumar(*args):
    total = 0

    for numero in args:
        total += numero

    return total

print(sumar(1, 2)) #3
print(sumar(1, 2, 3, 4)) #10
```

`**kwargs`

`**kwargs` permite recibir una cantidad variable de argumentos con nombre.

Internamente, `kwargs` es un diccionario.

```
def crear_perfil(**kwargs):
    for clave, valor in kwargs.items():
        print(f"{clave}: {valor}")
```

```
crear_perfil(nombre="Sebastian", edad=30, ciudad="Santiago")

"""
Entrega
nombre: Sebastian
edad: 30
ciudad: Santiago
"""
```

Variables locales y globales

El scope es el alcance de una variable. Indica donde existe y donde se puede usar.

Una variable creada dentro de una funcion es local. Solo existe dentro de esa funcion.

```
def ejemplo():
    mensaje = "Hola"
    print(mensaje)

ejemplo() #Hola
print(mensaje) #name 'mensaje' is not defined
```

global

Aunque funciona, usar `global` suele ser una mala practica porque hace que el codigo sea mas dificil de entender y mantener.

Es mejor usar `return`.

```
def aumentar(contador):
    return contador + 1

contador = 0
contador = aumentar(contador)

print(contador) #R: 1
```

Funciones que modifican en el lugar

Algunas funciones modifican directamente el objeto recibido y devuelven `None`.

`random.shuffle()` mezcla una lista en el lugar.

```
import random

lista = [1, 2, 3]
```

```
random.shuffle(lista)

print(lista) # La lista queda desordenada
```

No se debe guardar el resultado de `shuffle()`.

```
lista = random.shuffle(lista)
print(lista) #None
```

Ejemplo correcto:

```
def shuffle_alt(pregunta):
    random.shuffle(pregunta["alternativas"])
    return pregunta["alternativas"]
```

Si se escribe:

```
pregunta["alternativas"] = random.shuffle(pregunta["alternativas"])
```

primero se mezcla la lista, pero despues `pregunta["alternativas"]` queda valiendo `None`.

11. Listas

Las listas guardan varios elementos y se accede a ellos por posicion.

```
frutas = ["Manzana", "Pera", "Naranja"]

print(frutas)
# R: ['Manzana', 'Pera', 'Naranja']

print(frutas[0])
# R: Manzana
print(frutas[-1])
# R: Naranja

frutas.append("Platano")

"""
R:
Manzana
Pera
Naranja
Platano
"""
```

```
for fruta in frutas:
    print(fruta)

print(", ".join(frutas))
# R: Manzana, Pera, Naranja, Platano
```

Metodos y operaciones utiles:

Ejemplo	Resultado o uso
<code>frutas[0]</code>	Primer elemento.
<code>frutas.append("kiwi")</code>	Agrega al final.
<code>frutas.insert(1, "platano")</code>	Agrega en la posicion 1.
<code>frutas.pop(2)</code>	Elimina y devuelve el elemento eliminado.
<code>frutas.remove("uva")</code>	Elimina ese valor.
<code>frutas.reverse()</code>	Invierte la lista.
<code>frutas.sort()</code>	Ordena la lista original.
<code>sorted(numeros)</code>	Devuelve una copia ordenada.
<code>frutas.index("pera")</code>	Devuelve la posicion.
<code>max(numeros)</code>	Mayor valor.
<code>min(numeros)</code>	Menor valor.
<code>sum(numeros)</code>	Suma total.
<code>[1, 2] + [3, 4]</code>	<code>[1, 2, 3, 4]</code>
<code>[1, 2] * 3</code>	<code>[1, 2, 1, 2, 1, 2]</code>

Argumentos desde terminal

Con `sys.argv` se pueden leer argumentos escritos al ejecutar un archivo.

```
import sys

print(sys.argv[0])
print(sys.argv[1])

"""
Si en terminal se ejecuta:

python argumentos.py Sebastian

El resultado es:

argumentos.py
```



```
Sebastian  
"""
```

12. Tuplas

Las tuplas se parecen a las listas, pero normalmente se usan para datos que no cambian.

```
coordenadas = (10.0, 20.0)  
  
print(coordenadas)
```

Resultado:

```
(10.0, 20.0)
```

13. Sets

Los sets no permiten elementos duplicados.

```
numeros = {1, 1, 2, 3, 4, 5}  
  
print(numeros)
```

Resultado:

```
{1, 2, 3, 4, 5}
```

14. Diccionarios

Los diccionarios guardan informacion usando claves.

```
persona = {  
    "nombre": "Sebastian",  
    "edad": 30,  
    "profesion": "Ingeniero",  
}  
  
print(persona["nombre"])  
print(persona["edad"])  
  
transaction = {  
    "amount": 5000,
```

```

    "currency": "USD",
    "country": "CL",}

print(transaction)

"""
Resultados
Sebastian
30
{'amount': 5000, 'currency': 'USD', 'country': 'CL'}
"""

```

Ejemplo de lista con diccionarios:

```

transactions = [
    transaction1,
    transaction2,
    transaction3,
]

```

Ejemplo de comprehension con diccionarios:

```

ventas = {
    "enero": 100,
    "febrero": 250,
    "marzo": 80,
}

umbral = 100

ventas_mayores = {
    mes: monto
    for mes, monto in ventas.items()
    if monto > umbral
}

print(ventas_mayores)

# Resultado {'febrero': 250}

```

Operaciones comunes:

Ejemplo	Uso
<code>persona["peso"] = 72</code>	Agrega una clave.
<code>persona["edad"] = 31</code>	Modifica un valor.
<code>del persona["peso"]</code>	Elimina una clave.

Ejemplo	Uso
<code>persona.pop("peso")</code>	Elimina y devuelve el valor.
<code>a.update(b)</code>	Mezcla dos diccionarios.
<code>persona.keys()</code>	Entrega las claves.
<code>persona.values()</code>	Entrega los valores.
<code>persona.items()</code>	Entrega claves y valores.

Uso de `get()`:

```
persona = {  
    "nombre": "Sebastian",  
    "edad": 30,  
}  
  
print(persona.get("telefono"))  
# Entrega None  
  
print(persona.get("telefono", "No existe"))  
# Entrega No existe
```

Idea clave:

En una lista busco por posicion.
En un diccionario busco por clave.

15. Librerías

Las librerías permiten usar código ya hecho.

```
import math  
from math import ceil, pi, sqrt  
import math as m  
  
print(math.sqrt(25))  
print(math.ceil(3.14))  
print(math.pi)  
  
ceil_value = ceil(3.14)  
  
print(sqrt(25))  
print(ceil_value)  
print(pi)  
print(m.sqrt(25))
```

Resultado:

```
5.0
4
3.141592653589793
5.0
4
3.141592653589793
5.0
```

Instalacion de librerias externas:

```
pip install pandas
pip install numpy
```

16. Modularizacion

Modularizar es separar un programa en partes mas pequeñas y ordenadas.

Importaciones

Las importaciones permiten usar codigo desde otro modulo.

`import modulo`

Importa el modulo completo.

```
import math

print(math.sqrt(25)) #5.0
```

`import modulo as alias`

Importa el modulo con un nombre mas corto.

```
import math as m

print(m.sqrt(25)) #5.0
```

`from modulo import funcion`

Importa solo una funcion o elemento del modulo.

```
from math import sqrt

print(sqrt(25)) #5.0
```

main.py

main.py normalmente es el punto de entrada del programa.

Desde ahi se coordinan los demas modulos.

```
from calculadora import sumar

print(sumar(2, 3)) #5
```

```
if __name__ == "__main__":
```

Esto evita que un modulo ejecute codigo automaticamente al importarlo.

```
def saludar():
    print("Hola")

if __name__ == "__main__":
    saludar()
```

Docstrings

Los docstrings son textos que documentan que hace una funcion, clase o modulo.

```
def sumar(a, b):
    """
    Suma dos numeros.
    """
```

Refactorizacion

Refactorizar es mejorar el codigo sin cambiar su funcionamiento.

Antes:

```
print("Hola Ana")
print("Hola Luis")
```

Despues:

```
for nombre in ["Ana", "Luis"]:
    print(f"Hola {nombre}")
```

17. Experiencia de usuario

`time.sleep()`

`time.sleep()` pausa el programa por una cantidad de segundos.

```
import time

print("Cargando...")
time.sleep(2)
print("Listo")
```

Limpiar pantalla

Limpiar pantalla depende del sistema operativo.

```
os.system("cls")
os.system("clear")
```

`exit()`

`exit()` finaliza el programa.

```
exit()
```

18. Programacion orientada a objetos

La programacion orientada a objetos permite organizar datos y acciones relacionadas dentro de clases.

Concepto	Que significa
Clase	Molde que define las características y acciones de un tipo de objeto.
Objeto o instancia	Elemento concreto creado a partir de una clase.
Atributo	Característica o dato perteneciente a una clase u objeto.
Metodo	Funcion definida dentro de una clase.

Clase, atributo y metodo estatico

```
# CLASE:
# Es el molde que se utiliza para crear objetos de tipo Te.
# Los nombres de clases se escriben en PascalCase.
class Te:

    # ATRIBUTO DE CLASE:
    # Es una caracteristica compartida por todos los objetos de tipo Te.
    # Se usa de esta forma porque todos los tipos de te duran 365 dias.
    duracion = 365

    # DECORADOR:
    # Indica que obtener_precio es un metodo estatico.
    # Permite usar el metodo directamente desde la clase,
    # sin crear primero un objeto de tipo Te.
    @staticmethod

    # METODO ESTATICO:
    # Es una funcion definida dentro de una clase.
    # Recibe el formato del te y retorna su precio.
    def obtener_precio(formato):

        # Comprueba si el formato ingresado es de 300 gramos.
        if formato == 300:

            # Retorna el precio del formato de 300 gramos.
            return 3000

        # El ejercicio permite asumir que la otra opcion es 500 gramos.
        else:

            # Retorna el precio del formato de 500 gramos.
            return 5000
```

Crear un objeto

```
# OBJETO O INSTANCIA:
# Te() crea un objeto concreto utilizando la clase Te como molde.
# te_negro guarda ese objeto para poder utilizarlo posteriormente.
te_negro = Te()

# Accede al atributo de clase mediante el objeto.
print(te_negro.duracion)

# Tambien se puede acceder directamente desde la clase.
print(Te.duracion)
```

Ambos `print()` muestran 365 porque `duracion` es compartido por todas las instancias de `Te`.

Llamar un metodo estatico

```
# El metodo se llama desde la clase porque utiliza @staticmethod.  
# El argumento 300 se almacena en el parametro formato.  
precio = Te.obtener_precio(300)  
  
# Muestra el valor retornado por el metodo.  
print(precio)
```

Resultado:

```
3000
```

Metodo que retorna varios valores

Un metodo puede retornar mas de un valor. Estos se pueden guardar en variables separadas.

```
class Te:  
  
    @staticmethod  
    def obtener_datos(sabor):  
        if sabor == 1:  
            return 3, "Se recomienda consumir al desayuno"  
  
# Cada valor retornado se almacena en una variable.  
tiempo, recomendacion = Te.obtener_datos(1)  
  
print(tiempo)  
print(recomendacion)
```

Resultado:

```
3  
Se recomienda consumir al desayuno
```

Importar una clase

Si la clase se encuentra en otro archivo, debe importarse antes de utilizarla.

```
# Archivo te.py  
class Te:  
    duracion = 365
```



```
# Archivo pedido.py

# te corresponde al archivo te.py.
# Te corresponde a la clase que se desea importar.
from te import Te

pedido = Te()
print(pedido.duracion)
```

Estructura basica para recordar:

```
class NombreClase:
    atributo_de_clase = valor

    @staticmethod
    def nombre_metodo(parametro):
        return resultado

objeto = NombreClase()
resultado = NombreClase.nombre_metodo(argumento)
```